GFAz: Order-of-Magnitude Pangenome Analytics by Computing over Compression

Taolue Yang[1], Youyuan Liu[1], Bo Jiang[1], Xinghua Shi[1], and Sian Jin[1]

[1]Temple University, Philadelphia, PA, USA

Overview. Pangenome graphs in the Graphical Fragment Assembly (GFA) format now reach hundreds of gigabytes, dominated by haplotype traversal records (over 90% of file size). We present GFAz, which plays two roles over one container. As a *compressor*, it is lossless and reaches 18 to 84× compression, 15 to 20× better than gzip on the largest whole-genome graphs and well ahead of specialized tools. As a *compute engine*, it runs path-centric analyses directly on the compressed traversals, giving 25 to 613× speedups over standard downstream tools with far less memory, and scales to 369 GB whole-genome HPRC graphs the baselines cannot process at all.

Design. One principle underlies both halves: keep haplotype traversals in a grammar-compressed form, and both store and query them in that form.

Compression.

- **Orientation-aware grammar.** Paths and walks are normalized to a single signed `int32` traversal form (sign = strand). Each adjacent 2-mer is canonicalized so its forward and reverse-complement occurrences share one rule, and the rule's sign records the observed orientation; one rulebook thus serves both strands with no reverse copy.
- **Hierarchical multi-round reduction.** Each round replaces frequent adjacent 2-mers with new rule symbols, shrinking the stream by up to half; later rounds pair rules from earlier rounds, so the default 8 rounds capture exponentially longer haplotype repeats (length 4, 8, 16, ...). Paths and walks are mined jointly, and every round is a single linear scan, making compression and decompression linear-time.
- **Columnar, fully lossless layout.** All record types, including optional fields, are stored as type-specific columns, enabling per-field access, incremental haplotype extension, and single-haplotype random access.
- **Parallel CPU/GPU backends.** The whole pipeline is fully parallel, with interchangeable OpenMP (CPU) and CUDA/nvComp (GPU) backends that share one format.

Compute engine.

- **Materialization-free execution.** Queries run directly on the grammar-compressed traversal store: GFAz never rebuilds the GFA text, a whole-graph object, or even a fully expanded list of traversal steps. This avoids the up-front cost that graph- and text-resident tools (vg, odgi, Panacus) pay by materializing a representation larger than the statistic they compute.
- **Unified decode-and-fold execution model.** A shared primitive expands one traversal to orientation-signed node IDs straight from the grammar (array-indexed rule lookup, no hashing) and folds it into a compact accumulator under a parallel loop. A query touches only the columns it needs, so peak memory stays below the uncompressed GFA and every command is the same decode-and-fold skeleton differing only in its accumulator.
- **Two-phase GFA-to-VCF projection (`deconstruct`).** A new algorithm that separates one-time, topology-only site discovery (biconnected decomposition of the node-end graph, $O(V+E)$) from a streaming allele-observation fold, never building a graph or a snarl index.

- **Generality across analyses.** `growth`, `pav`, `depth`, `similarity`, and `stats` reuse the same substrate, each reproducing an established tool's output.

Compression performance. All results use a single node with an AMD Ryzen Threadripper PRO 9955WX (16 cores, 32 threads), an NVIDIA RTX Pro 6000 GPU, and 512 GB of DDR5-6400 memory (503 GiB usable). Across six datasets (113 MB to 369 GB), GFAz gives the best ratio of every method on both backends and also leads compression and decompression throughput where inputs fit: the CPU backend sustains up to 385 MB/s and 1.8 GB/s, and the GPU backend up to 4.8 GB/s and 9.4 GB/s. The margin over specialized tools widens with graph complexity: on HPRC v2.0 GFAz reaches 83.9× versus 66.8× (GBZ) and 6.5× (zstd), while sqz trails by about 2× at chromosome scale and runs out of memory on every HPRC graph (Table 1).

Table 1: Compression ratio and throughput (MB/s) vs. baselines. Ratio = ×; Co. = compression speed, De. = decompression speed. **Bold** = best in row. "sqz" is the best-ratio sqz variant (sqz+bgzip); it runs out of memory on all HPRC graphs (NA). GPU throughput on the two largest graphs is omitted (traversals exceed device memory; ratio shown via CPU simulation). GBZ and sqz preserve fewer GFA semantics.

Dataset (GFA)		Gzip	Zstd	sqz	GBZ	GFAz (CPU)	GFAz (GPU)
chr1 (7.1 GB)	Ratio	5.59	7.54	18.0	9.52	**35.4**	31.7
	Co.	46	2178	4.0	12	385	**2754**
	De.	359	1618	21	284	1658	**8124**
chr6 (3.8 GB)	Ratio	5.04	6.99	20.8	19.2	**35.4**	28.2
	Co.	41	1712	3.6	11	348	**3791**
	De.	348	1515	20	281	1758	**7230**
E. coli (113 MB)	Ratio	4.69	5.67	7.46	5.58	**18.3**	16.7
	Co.	33	**1356**	4.5	20	190	678
	De.	310	1258	32	197	491	**1430**
HPRC v1.1 (48 GB)	Ratio	4.02	5.32	NA	14.0	**22.4**	20.4
	Co.	36	1657	NA	85	231	**4843**
	De.	319	1234	NA	650	1058	**9435**
HPRC v2.0 (358 GB)	Ratio	4.19	6.49	NA	66.8	**83.9**	76.4
	Co.	49	**1514**	NA	130	367	n/a
	De.	342	1240	NA	648	**1652**	n/a
HPRC v2.1 (369 GB)	Ratio	4.19	6.43	NA	64.2	**82.8**	74.2
	Co.	49	**1540**	NA	136	348	n/a
	De.	343	1241	NA	652	**1559**	n/a

Compute performance. Computing on the container matches each tool's output while running far faster on far less memory (Table 2): `deconstruct` is 50 to 71× faster than `vg` at 3.3× less peak RSS, `growth` is 25 to 230× faster than Panacus at 7 to 16× less RSS, and `pav` is 299 to 613× faster than `odgi pav` at about 3× less RSS, with outputs in agreement (about 99.99% of `deconstruct` positions, identical growth curves and PAV semantics). This sub-GFA footprint is what enables the whole-genome scaling: from a 4.5 GB container GFAz emits the genome-wide VCF (about 35 M records) on the 358 and 369 GB HPRC v2 graphs in about 40 min at 191 to 195 GB RSS (roughly half the GFA), and the exact growth curve in about 40 s at only 13 to 14 GB.

Table 2: Downstream analytics vs. standard tools (16 threads): wall-clock time and peak RSS. GFAz reads the compressed `.gfaz`; baselines read the GFA. "DNF" = baseline does not complete in practical memory, so GFAz runs alone. (odgi `pav` additionally needs an `odgi build` step, 31–61 s; whole-genome `pav` on the two largest graphs exceeds the 503 GB node and is omitted.)

		GFAz		Baseline		
Analysis	Graph (GFA)	Time	RSS	Time	RSS	Speedup
`deconstruct` vs. `vg`	chr1 (7.1 GB)	11.3 s	8.3 GB	805 s	30.3 GB	71×
	chr6 (3.8 GB)	7.0 s	5.2 GB	349 s	18.4 GB	50×
	HPRC v1.1 (48 GB)	5.1 min	37.4 GB	DNF	n/a	n/a
	HPRC v2.0 (358 GB)	39.8 min	191 GB	DNF	n/a	n/a
	HPRC v2.1 (369 GB)	41.4 min	195 GB	DNF	n/a	n/a
`growth` vs. Panacus	chr1 (7.1 GB)	0.74 s	0.66 GB	18.3 s	6.16 GB	25×
	chr6 (3.8 GB)	0.36 s	0.31 GB	9.0 s	3.77 GB	25×
	HPRC v1.1 (48 GB)	6.2 s	6.3 GB	1428 s	45.8 GB	230×
	HPRC v2.0 (358 GB)	39.8 s	12.9 GB	DNF	n/a	n/a
	HPRC v2.1 (369 GB)	41.2 s	13.7 GB	DNF	n/a	n/a
`pav` vs. `odgi`	chr1 (7.1 GB)	11.7 s	9.5 GB	3499 s	31.9 GB	299×
	chr6 (3.8 GB)	7.8 s	6.4 GB	4759 s	19.4 GB	613×
	HPRC v1.1 (48 GB)	1.8 min	77 GB	DNF	n/a	n/a

Conclusion. GFAz folds the two recurring costs of large pangenomes, storage and derivation, into a single container that is state-of-the-art at compression and a fast, low-memory engine for the analyses practitioners run in practice. It is open source at `https://github.com/babyplutokurt/GFAz`.